FILE: C:\HCL_TRAINING\client\src\context\AuthContext.jsx

LINES: 105 | SIZE: 2.8 KB | LANGUAGE: JSX

MODULE: Authentication Context with Session Restoration & Auto-Login

```
 1 | import React, { createContext, useContext, useState, useEffect } from 'react';
 2 | import { api } from '../api/client';
 3 |
 4 | const AuthContext = createContext(null);
 5 |
 6 | export const AuthProvider = ({ children }) => {
 7 |   const [user, setUser] = useState(() => {
 8 |     const saved = localStorage.getItem('hiresense_user');
 9 |     return saved ? JSON.parse(saved) : null;
10 |   });
11 |   const [token, setToken] = useState(() => localStorage.getItem('hiresense_token') || nu
   > ll);
12 |   const [loading, setLoading] = useState(true);
13 |
14 |   useEffect(() => {
15 |     const checkAuth = async () => {
16 |       const savedToken = localStorage.getItem('hiresense_token');
17 |       if (savedToken) {
18 |         try {
19 |           const res = await api.get('/auth/me');
20 |           if (res.success && res.data) {
21 |             setUser(res.data);
22 |             localStorage.setItem('hiresense_user', JSON.stringify(res.data));
23 |           }
24 |         } catch (err) {
25 |           console.warn('Session check failed or expired:', err.message);
26 |           setToken(null);
27 |           setUser(null);
28 |           localStorage.removeItem('hiresense_token');
29 |           localStorage.removeItem('hiresense_user');
30 |         }
31 |       }
32 |       setLoading(false);
33 |     };
34 |
35 |     checkAuth();
36 |   }, []);
37 |
38 |   const login = async (email, password) => {
39 |     const res = await api.post('/auth/login', { email, password });
40 |     if (res.success && res.token) {
41 |       setToken(res.token);
42 |       setUser(res.user);
43 |       localStorage.setItem('hiresense_token', res.token);
44 |       localStorage.setItem('hiresense_user', JSON.stringify(res.user));
45 |     }
46 |     return res;
47 |   };
48 |
49 |   const register = async (userData) => {
50 |     const res = await api.post('/auth/register', userData);
51 |     if (res.success && res.token) {
52 |       setToken(res.token);
53 |       setUser(res.user);
54 |       localStorage.setItem('hiresense_token', res.token);
55 |       localStorage.setItem('hiresense_user', JSON.stringify(res.user));
56 |     }
57 |     return res;
58 |   };
59 |
60 |   const logout = async () => {
61 |     try {
62 |       await api.post('/auth/logout', {});
63 |     } catch (e) {
64 |       // ignore
65 |     }
```

```
66 |       setToken(null);
67 |       setUser(null);
68 |       localStorage.removeItem('hiresense_token');
69 |       localStorage.removeItem('hiresense_user');
70 |     };
71 |
72 |     const updateUser = (updated) => {
73 |       setUser((prev) => {
74 |         const merged = { ...prev, ...updated };
75 |         localStorage.setItem('hiresense_user', JSON.stringify(merged));
76 |         return merged;
77 |       });
78 |     };
79 |
80 |     return (
81 |       <AuthContext.Provider
82 |         value={{
83 |           user,
84 |           token,
85 |           isAuthenticated: !!user,
86 |           loading,
87 |           login,
88 |           register,
89 |           logout,
90 |           updateUser,
91 |         }}
92 |       >
93 |         {children}
94 |       </AuthContext.Provider>
95 |     );
96 |   };
97 |
98 |   export const useAuth = () => {
99 |     const context = useContext(AuthContext);
100 |     if (!context) {
101 |       throw new Error('useAuth must be used within an AuthProvider');
102 |     }
103 |     return context;
104 |   };
105 |
```